HyperWave: A Peer-to-Peer Stadium Wave with Burned-Fee Economics

Tether Developers Cup 2026 · github: mnaamani/hyper-wave

ABSTRACT. HyperWave recreates the stadium "Mexican wave" as a serverless, permissionless protocol. Peers derive a fixed seat on a logical ring from their public key, a signed token races seat-to-seat leaving a constant-size cryptographic trace, and each participant's selfie converges into a replicated, multi-writer gallery. Real (testnet) value flows through the game with no beneficiary or operator: participation fees are provably *burned* on-chain (anti-spam) and viewers tip selfies wallet-to-wallet. The system runs on the Holepunch stack (Hyperswarm, Autobase) with a Chord overlay for scale, and Tether's WDK for self-custodial payments. Every peer runs identical code; there are no roles.

1 Introduction

A stadium wave is a self-organising relay: thousands of people produce a global pattern from one local rule — *react to your neighbour, then pass it on*. HyperWave moves this phenomenon onto the open Internet. The design goals are (i) **no servers** — discovery, state, and money are all peer-to-peer; (ii) **no trust** — every claim a peer makes is signed and cheaply verifiable; (iii) **no free spam** — starting or joining a wave costs a small, provable, beneficiary-less fee; and (iv) **equality** — every peer runs the same code, with no privileged validator or seed node.

2 The ring: identity is the seat

Each peer holds an Ed25519 keypair. Its ring position (angle) is derived from the top bytes of the public key, so a seat can be neither chosen nor forged, and grinding keys buys nothing but a different random seat. Peers on the same *match topic* discover each other via Hyperswarm's DHT and communicate over Noise-encrypted streams. A peer's **successor** is the next live peer clockwise; liveness rides a small heartbeat that also carries Chord pointers (§6).

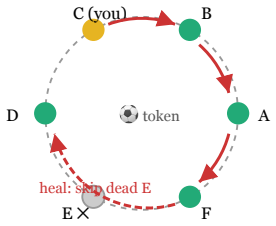

Fig. 1. Seats derived from keys; the token visits successors clockwise and heals around a dead peer (E).

3 The wave

A wave has a lifecycle of *idle → lobby → racing → idle*, one wave at a time. Any peer may kick off: it pays the fee (§5), then floods a signed announcement opening a short **lobby** in which peers opt in (paying their own fee) and stage a webcam selfie — captured *before* the race, so the token never waits for a human. The initiator then starts the race.

The **token** hops from each holder to its successor. Every holder signs a *receipt* over `(waveId, hop, chain, t)` and folds it into a rolling accumulator carried by the token:

$$\texttt{chain}' = \texttt{BLAKE2b}(\texttt{chain} \parallel \texttt{sig}_{\texttt{hop}})$$

The token therefore stays a few hundred bytes at any crowd size, yet commits to the full ordered history of who carried the wave (Fig. 2). Anyone holding the receipts can replay the fold and verify the lap. If a successor fails to acknowledge within a timeout, the holder skips to the next live seat; when the token returns to the initiator, a signed *wave-end* is flooded and all peers finish together.

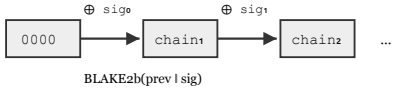

Fig. 2. The constant-size receipt-chain accumulator: each hop's signature is folded into a rolling hash.

4 The gallery

Each wave mints a fresh multi-writer log (an Autobase keyed by the random wave id; its key is signed by the initiator so relays cannot substitute a rogue log). As the token reaches a participant, their staged selfie is appended, and the gallery fills in seat order on every screen. Writes are gated twice: admission requires a valid hop receipt plus a signed fee attestation, and a deterministic apply rule — identical on every peer — verifies each entry's signature, enforces *one entry per peer* and byte caps, and keeps a tip address only if the signed fee names that wallet. Galleries are ephemeral by design; the wave's initiator retains its own wave's gallery for latecomers while it remains online.

5 Economics: burn, tip

Every peer embeds a self-custodial wallet (WDK; Tron Nile testnet, native TRX). Two rules define the entire economy:

- **Fees are burned.** Starting or joining costs 1 TRX sent to Tron's unspendable black-hole address, memo-tagged `hyperwave:<wave>:<peer>`. Nobody profits; spam costs real money. Peers verify the initiator's burn on-chain before joining — an unpaid wave is ignored by all.
- **Tips are the only earnings.** A viewer tips a selfie 1 TRX directly to its owner's wallet; the apply rule guarantees a tip can only reach a wallet that paid in.

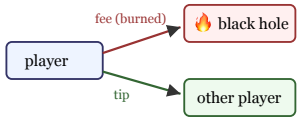

Fig. 3. Money flows: fees are destroyed, tips go peer-to-peer.

6 Scaling and security

Topology. Small swarms full-mesh, but the design does not depend on it: the ring *drives* connections. Each peer pins its successor list (k=3), predecessor, and O(log N) Chord fingers, and a distributed `findSuccessor` RPC resolves lookups correctly under partial membership knowledge. Lifecycle messages are flooded epidemically (relay on first sight of a message id), blanketing a partial mesh in a few rounds. A serial lap is O(N) in wall clock; a deterministic timed sweep — each seat self-triggering from a flooded start time — is the designed path to arbitrarily-large instant waves.

Adversarial notes. Self-describing sender fields must match the authenticated connection identity; flooded messages are instead authenticated by the signatures they carry. Wave termination is signed by the originator. Gallery admission is *optimistic*: the fee attestation's signature is checked locally, and the on-chain read is deferred to where it has value — tip time — bounding a fake burn's payoff at zero while keeping the write path free of chain I/O.

Wire surface. The protocol is deliberately small: one encoding (signed JSON over a single multiplexed channel) and ten message kinds, each assigned the cheapest propagation class that meets its need (Table 1).

Table 1. Message kinds and how each propagates.

MESSAGE	PROPAGATION & ROLE
`pointers`	neighbour-scoped heartbeat: liveness, country, Chord successor/predecessor
`wave-announce / join / start / end`	epidemic flood (dedup by message id): the wave lifecycle, paid-gate attested
`token`	unicast to the successor: the racing ball + receipt accumulator
`wave-pos`	one-hop broadcast: ball position for the visual roll; doubles as the heal ACK
`wave-sync`	unicast on connect: catch-up state for late joiners
`add-writer`	flood: gallery admission request, authenticated by its carried receipt
`find-succ / -reply`	routed via fingers: distributed successor lookup (Chord)

7 Gossip vs. replicated logs

HyperWave keeps two kinds of shared state and moves each with a different tool. The **control plane** — lifecycle, ball positions, admission requests — is ephemeral: it must reach everyone quickly, needs no history, and any peer may originate it. It travels as flooded gossip: unordered, deduplicated by message id, discarded once processed, with authenticity carried per-message by the signatures inside. The **data plane** — the gallery — is durable and ordered: it must converge to the same view everywhere and remain fetchable by latecomers. It lives in Autobase, Holepunch's multi-writer abstraction: each writer appends to its own signed Hypercore, and a deterministic *apply* function merges the inputs into one linearised view (the same family as Hyperbee, the append-only B-tree index over a single-writer Hypercore).

The planes could be collapsed — every gossip message an Autobase op — but that would buy ordering, causality, and persistence for messages that need none of it, at the cost of writer admission for every participant, per-wave log growth, and log-replication latency on the token's hot path. Conversely, running the gallery over bare gossip would forfeit convergence and late-join fetch. The split — transient flooding for *signals*, replicated logs for *artifacts* — is the design's central economy, and the admission handshake (a flooded, receipt-authenticated request that results in an Autobase writer grant) is the bridge between the two.

8 Potential applications

The substrate generalises beyond the game. The receipted token walk is a distributed round-robin with an audit trail — applicable to fair rotation and duty assignment among mutually distrusting parties, token-based mutual exclusion across trust boundaries, and roll-call liveness ceremonies (a completed lap proves N specific identities were live in a window). The ring gives every key a *place*, suggesting a rendezvous/metadata key-value service with per-record payment (the niche BitTorrent's Mainline DHT proved viable), and store-and-forward messaging with paid postage and paid mailboxes at `successor(recipient)` — the burn pattern directly answers the spam and carrier-incentive gaps that sank earlier peer-to-peer messengers.

9 Related work

Chord [1] supplied the ring and routing but its storage applications never left the testbed; the only DHT with mass deployment, BitTorrent's Mainline, succeeded with tiny, ephemeral, self-certifying values — conditions HyperWave's per-wave state deliberately shares. Gossip messengers (Scuttlebutt [8], Ethereum's Whisper [9]) repeatedly foundered on the same two gaps — no spam cost and no incentive to carry others' data — that Whisper's own successor (Waku [10]) later addressed with rate-limiting proofs and payments. HyperWave differs in making the economics native from the start: a wallet in every peer, verifiable burns as the admission ticket, and direct transfers as the reward path — while Hyperswarm supplies the NAT traversal whose absence sank much of the earlier P2P generation.

10 Limitations and future work

The serial lap is O(N) in wall clock — a spectacle for rooms and stadiums, not planets; the deterministic sweep (§6) is the designed successor for large N and drops only the hop-interlocked receipt chain, which no payment depends on. Galleries are ephemeral and a wave's gallery survives only while its initiator stays online; durable archival is a policy choice deliberately left out. The swarm identity is currently regenerated per run (the wallet persists; the seat does not) — persisting the keypair is queued work.

11 Status

Implemented and verified live on Nile: the ring, token race with healing, per-wave galleries, burned fees with on-chain verification, and tipping, running under a desktop Electron host. Correctness is covered by unit suites (ring/token/gallery/Chord/flood/pay), an 8-peer end-to-end harness on a local DHT including mid-race peer kills, and an on-chain end-to-end tier that drives a full paid wave with real burns. Neither the protocol nor the implementation has undergone an independent security audit; the adversarial analysis in §6 is the authors' own. Code and the full protocol specification are in the repository.

References

1. I. Stoica et al. Chord: A scalable peer-to-peer lookup service. *SIGCOMM*, 2001. pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf
2. A. Demers et al. Epidemic algorithms for replicated database maintenance. *PODC*, 1987. courses.cs.washington.edu/courses/cse552/07sp/papers/epidemic.pdf
3. Holepunch. Hyperswarm, Hypercore, Autobase. github.com/holepunchto
4. Tether. Wallet Development Kit (WDK). docs.wdk.tether.io
5. K. Karantias, A. Kiayias, D. Zindros. Proof-of-burn. *Financial Cryptography*, 2020. eprint.iacr.org/2019/1096
6. D. Boneh et al. Verifiable delay functions. *CRYPTO*, 2018. eprint.iacr.org/2018/601
7. I. Farkas, D. Helbing, T. Vicsek. Mexican waves in an excitable medium. *Nature* 419, 2002. nature.com/articles/419131a
8. D. Tarr et al. Secure Scuttlebutt: An identity-centric protocol for subjective and decentralized applications. *ACM ICN*, 2019. conferences.sigcomm.org/acm-icn/2019/proceedings/icn19-19.pdf
9. Ethereum. Whisper specification. EIP-627, 2017. eips.ethereum.org/EIPS/eip-627
10. Vac / Status. Waku: peer-to-peer messaging protocols. waku.org